

Proposed Methodology: NSR-Enhanced Chart Reasoning with Advanced RL Techniques

Abstract

This thesis proposes a novel approach to improve chart reasoning in Vision-Language Models (VLMs) by combining Negative Sample Reinforcement (NSR) with state-of-the-art reinforcement learning techniques. Building upon Chart-RVR's verifiable reward framework, we integrate five key innovations: (1) Dynamic Sampling from DAPO to filter uninformative gradients, (2) Selective Sample Replay (SSR) to prioritize high-value training samples, (3) Program-of-Thoughts (PoT) for verifiable symbolic reasoning, (4) Chain-of-Table for structured data manipulation, and (5) Self-Consistency for robust inference. We hypothesize that NSR's diversity-preserving properties, combined with these advanced techniques, will significantly improve out-of-distribution (OOD) performance on chart reasoning tasks while maintaining competitive in-domain accuracy.

1. Research Question

Primary Question:

Can Negative Sample Reinforcement (NSR), enhanced with modern RL optimizations and structured reasoning techniques, improve chart reasoning while maintaining output diversity, especially on out-of-distribution charts?

Sub-Questions:

1. How does NSR compare to standard GRPO in preserving reasoning diversity for chart tasks?
 2. Can Program-of-Thoughts and Chain-of-Table provide verifiable intermediate supervision that amplifies NSR's benefits?
 3. What is the optimal balance between positive and negative reinforcement (W-REINFORCE) for chart reasoning?
 4. Do diversity-preserving training methods improve OOD generalization on visually diverse chart datasets?
-

2. Background and Motivation

2.1 Current State of Chart Reasoning

Chart-RVR (Sinha et al., 2025) demonstrated that verifiable rewards improve chart reasoning:

- **In-domain (ChartQA):** 84.56% accuracy
- **Out-of-distribution (EvoChart):** 53.36% accuracy
- **Key innovation:** Multi-component rewards (format + chart type + table reconstruction + process conformity + accuracy)

However, the 31.2% performance gap between in-domain and OOD reveals brittleness when encountering unseen chart styles.

2.2 NSR Breakthrough in Math Reasoning

NSR (Zhu et al., 2025) showed that negative reinforcement alone matches or exceeds standard RL:

- **Pass@1:** 75.7% (slightly lower than PSR's 77.1%)
- **Pass@256:** 96.9% (significantly higher than PSR's 93.2%)
- **Key insight:** Penalizing incorrect answers preserves diversity while PSR causes distribution collapse

2.3 Why Charts Need Diversity

Chart reasoning presents unique challenges requiring diverse solution strategies:

1. **Visual Variations:** Different styles, colors, layouts, fonts
2. **Multiple Valid Reasoning Paths:** Type→Table→Calculate can be executed in various orders
3. **OOD Robustness:** Unseen chart styles require flexible interpretation strategies

Hypothesis: NSR's diversity preservation will transfer well to chart reasoning, improving OOD robustness.

3. Proposed Methodology

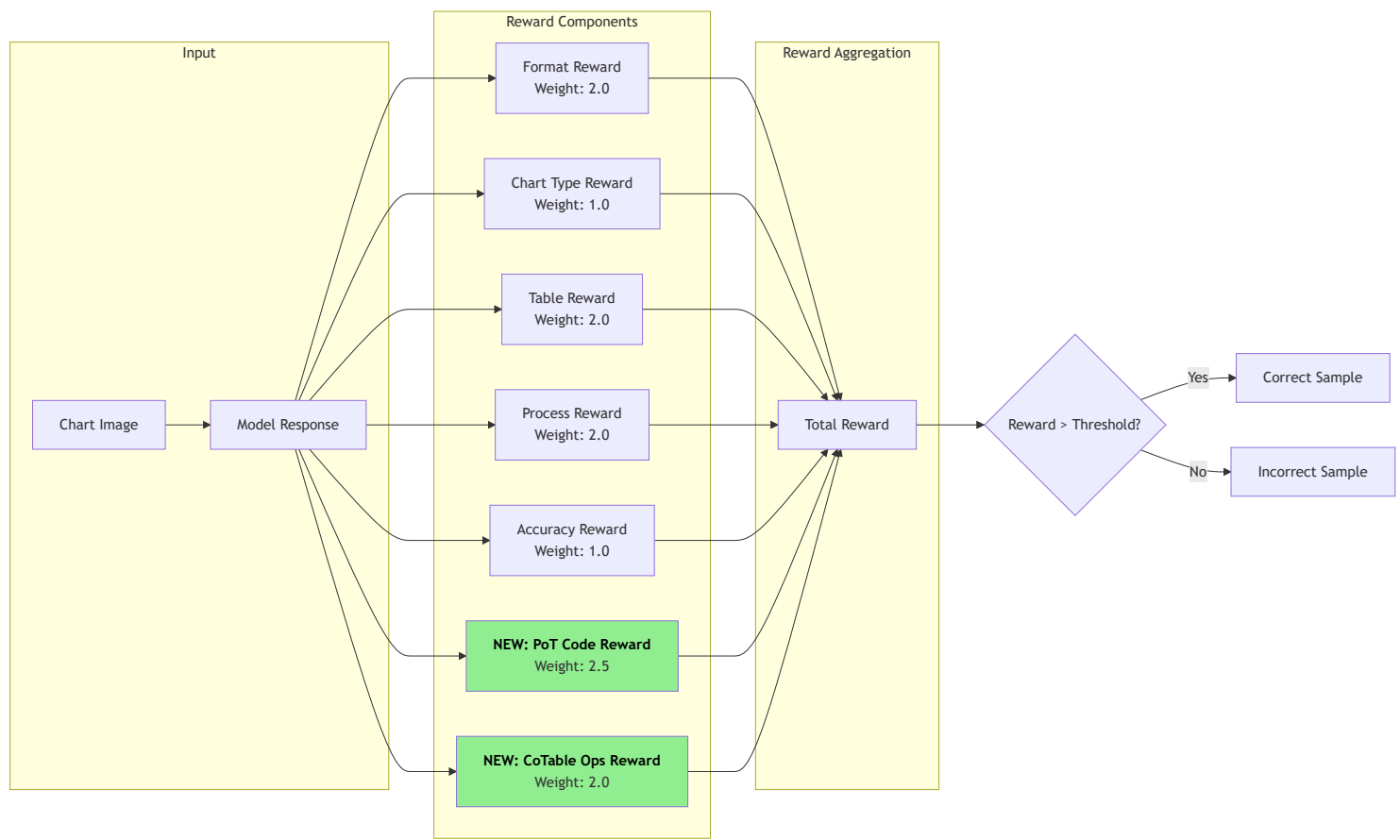
3.1 Overall Architecture



Syntax error in text
mermaid version 11.12.1

3.2 Enhanced Reward Architecture

We extend Chart-RVR's reward system with two new components:

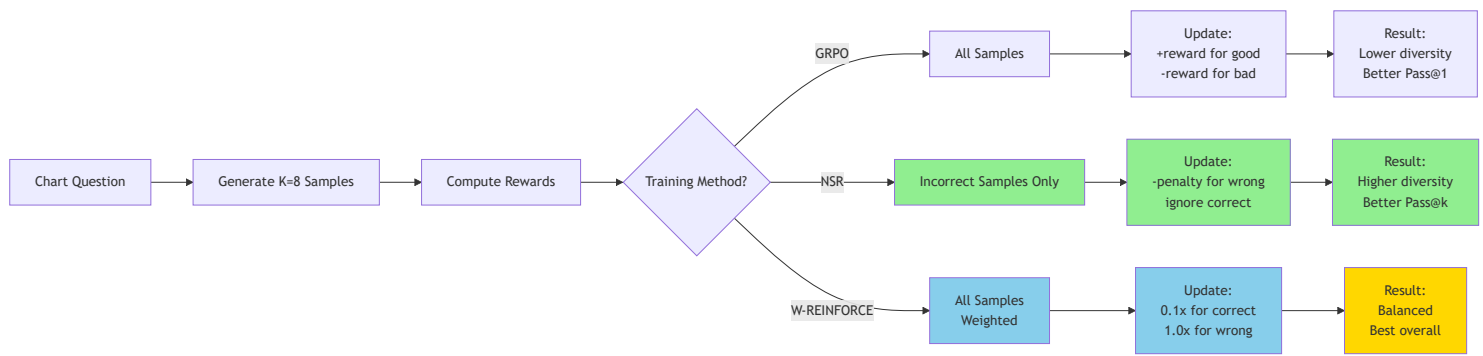


Reward Component Details

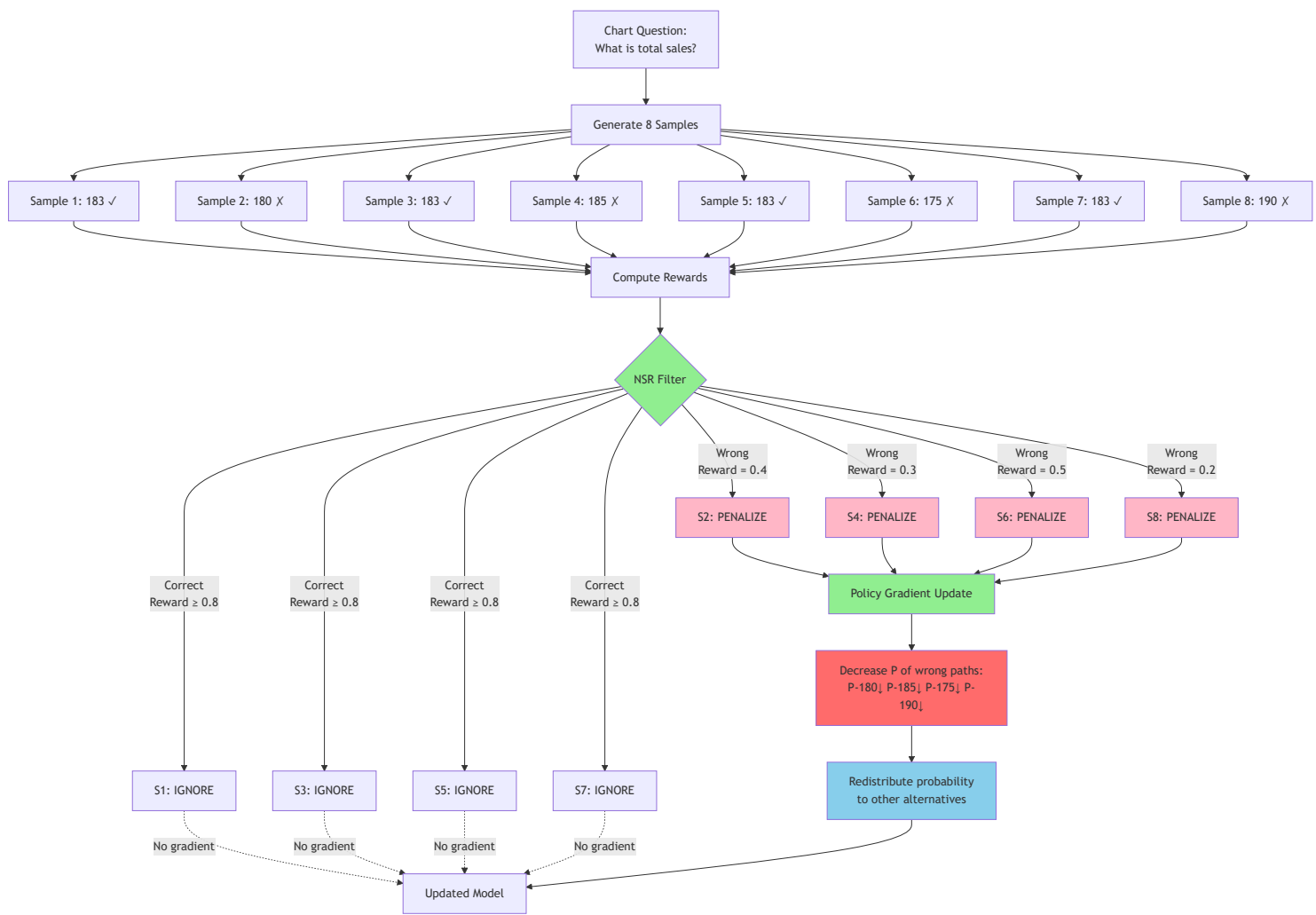
Component	Weight	Verification Method	Purpose
Format	2.0	Regex pattern matching	Ensures structured output
Chart Type	1.0	Classification accuracy	Visual understanding
Table Extraction	2.0	String similarity (edit distance)	Data comprehension
Process Conformity	2.0	Semantic similarity to gold CoT	Reasoning quality
Accuracy	1.0	Exact match or numerical tolerance	Correctness
PoT Code	2.5	Execution success + output correctness	Verifiable computation
CoTable Ops	2.0	Valid operation sequence + result	Structured manipulation

3.3 Training Method Comparison: GRPO vs NSR vs W-REINFORCE

High-Level Comparison



Detailed NSR Pipeline



Mathematical Formulation

GRPO Update:

```
For each sample i in {1..K}:
  advantage_i = reward_i - mean(rewards)
  loss += -advantage_i × log P(sample_i)
```

NSR Update (Ours):

```
For each sample i where reward_i < threshold:
  advantage_i = -(1 - reward_i) # Negative advantage
  loss += -advantage_i × log P(sample_i)

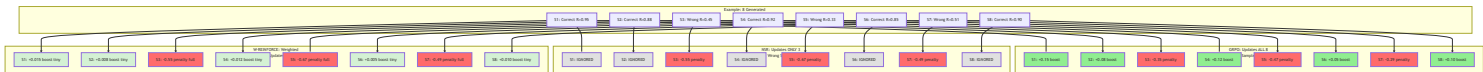
For each sample i where reward_i ≥ threshold:
  # Skip - no gradient update
```

W-REINFORCE Update:

```
λ = 0.1 # Positive weight

For each sample i:
  if reward_i ≥ threshold:
    advantage_i = λ × reward_i
  else:
    advantage_i = -(1 - reward_i)
  loss += -advantage_i × log P(sample_i)
```

Visual Comparison: What Gets Updated?



Key Insights:

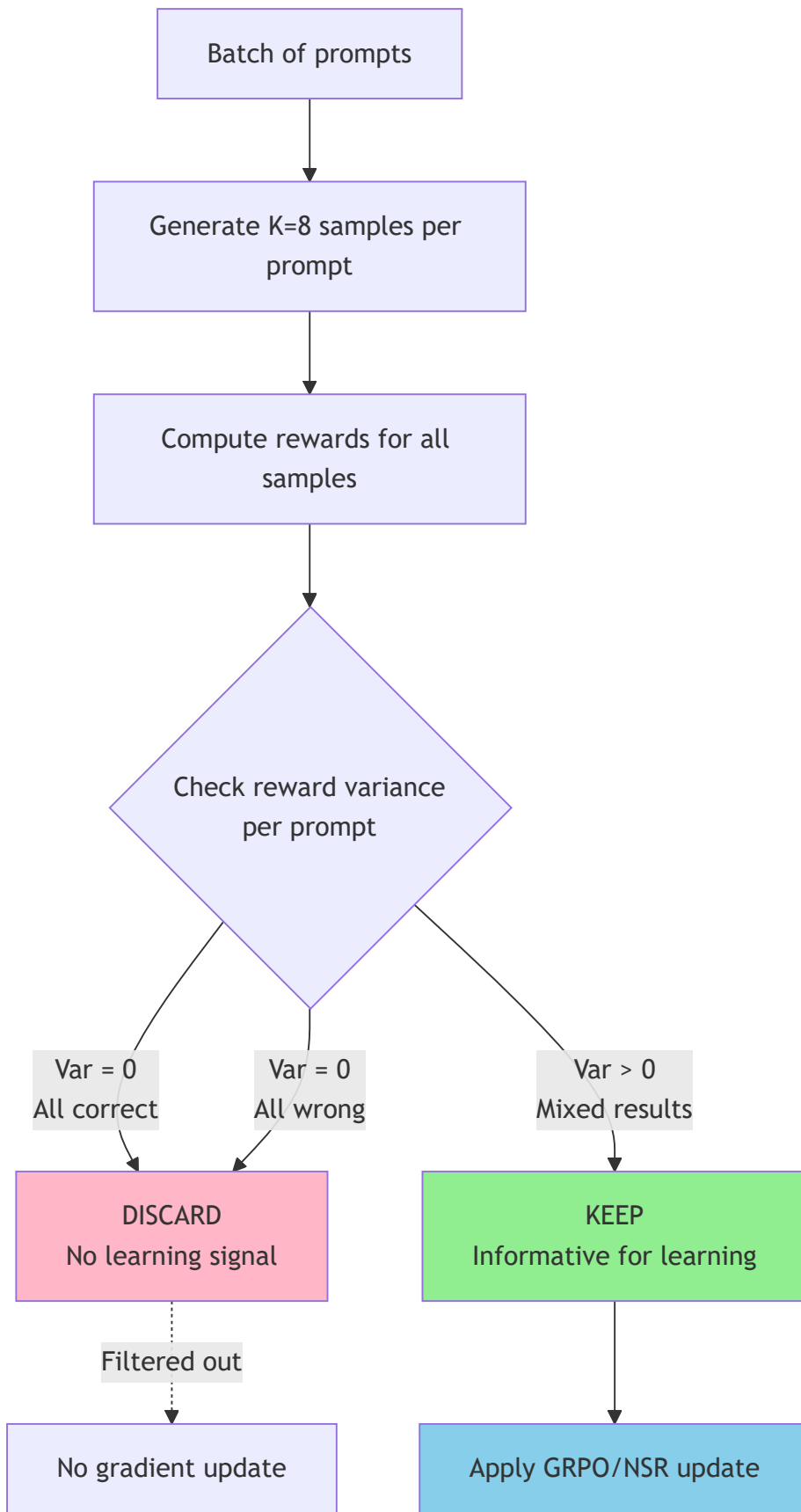
Method	Correct Samples	Wrong Samples	Effect on Diversity
GRPO	Strong positive boost	Strong negative penalty	✗ Collapses to single path
NSR	Completely ignored	Strong negative penalty	✓ Preserves multiple paths
W-REINFORCE	Weak positive boost (10%)	Strong negative penalty	⚖️ Balanced approach

Why NSR preserves diversity:

- By ignoring correct samples, NSR doesn't reinforce a single "correct" path
- Probability mass removed from wrong paths redistributes across ALL alternatives
- This maintains multiple valid reasoning strategies

3.4 DAPO Dynamic Sampling Integration

DAPO addresses GRPO's failure modes by filtering uninformative training batches:



Key Benefits:

- **Eliminates gradient noise** from trivially easy/hard samples
- **Focuses learning** on capability boundary

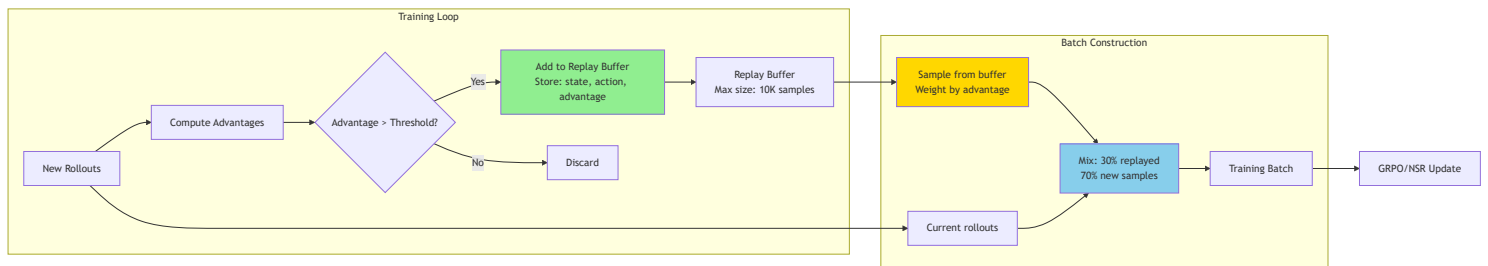
- **20-point improvement** on AIME 2024 (50% vs 30% vanilla GRPO)
- **50% fewer training steps** to convergence

Implementation:

```
# Pseudocode for Dynamic Sampling
def filter_batch(prompts, samples, rewards):
    informative_batches = []
    for prompt_idx in range(len(prompts)):
        prompt_rewards = rewards[prompt_idx]
        if np.var(prompt_rewards) > 0: # Mixed correctness
            informative_batches.append(prompt_idx)
    return informative_batches
```

3.5 Selective Sample Replay (SSR)

SSR maintains a replay buffer of high-advantage samples to combat vanishing advantages:



Key Benefits:

- **+3.2% improvement** on MathVista over vanilla GRPO
- **Preserves rare but valuable** learning signals
- **Prevents catastrophic forgetting** of edge cases
- **Minimal overhead:** Simple buffer management

Implementation:


```

class SelectiveSampleReplay:
    def __init__(self, max_size=10000, replay_ratio=0.3):
        self.buffer = []
        self.max_size = max_size
        self.replay_ratio = replay_ratio

    def add(self, sample, advantage):
        if advantage > threshold:
            self.buffer.append((sample, advantage))
            if len(self.buffer) > self.max_size:
                # Remove lowest advantage sample
                self.buffer.sort(key=lambda x: x[1])
                self.buffer.pop(0)

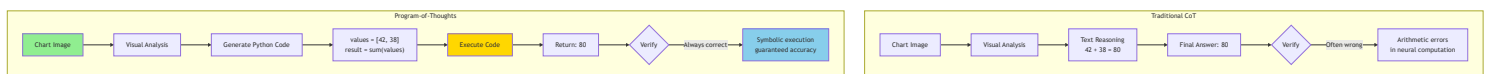
    def sample_batch(self, new_samples, batch_size):
        n_replay = int(batch_size * self.replay_ratio)
        n_new = batch_size - n_replay

        # Sample from buffer weighted by advantage
        replayed = weighted_sample(self.buffer, n_replay)
        return replayed + new_samples[:n_new]

```

3.6 Program-of-Thoughts (PoT) Integration

PoT offloads numerical computation to a Python interpreter, eliminating arithmetic errors:



Extended Response Format:

```

<think>
  <type>bar chart</type>
  <table>
    {"columns": ["Year", "Sales"], "rows": [[2020, 42], [2021, 38]]}
  </table>
  <reasoning>
    The question asks for total sales across both years.
    I need to sum the values in the Sales column.
  </reasoning>
  <code>
    # Extract sales values
    sales_2020 = 42
    sales_2021 = 38

    # Calculate total
    total = sales_2020 + sales_2021
    print(total)
  </code>
</think>
<answer>80</answer>

```

PoT Reward Computation:

```

def compute_pot_reward(response, ground_truth):
    # Extract code block
    code = extract_code(response)

    try:
        # Execute in sandboxed environment
        result = safe_execute(code, timeout=5)

        # Compare with ground truth
        if numerical_match(result, ground_truth):
            return 2.5 # Full reward
        elif partial_match(result, ground_truth):
            return 1.5 # Partial credit
        else:
            return 0.0
    except Exception:
        return 0.0 # Execution failure

```

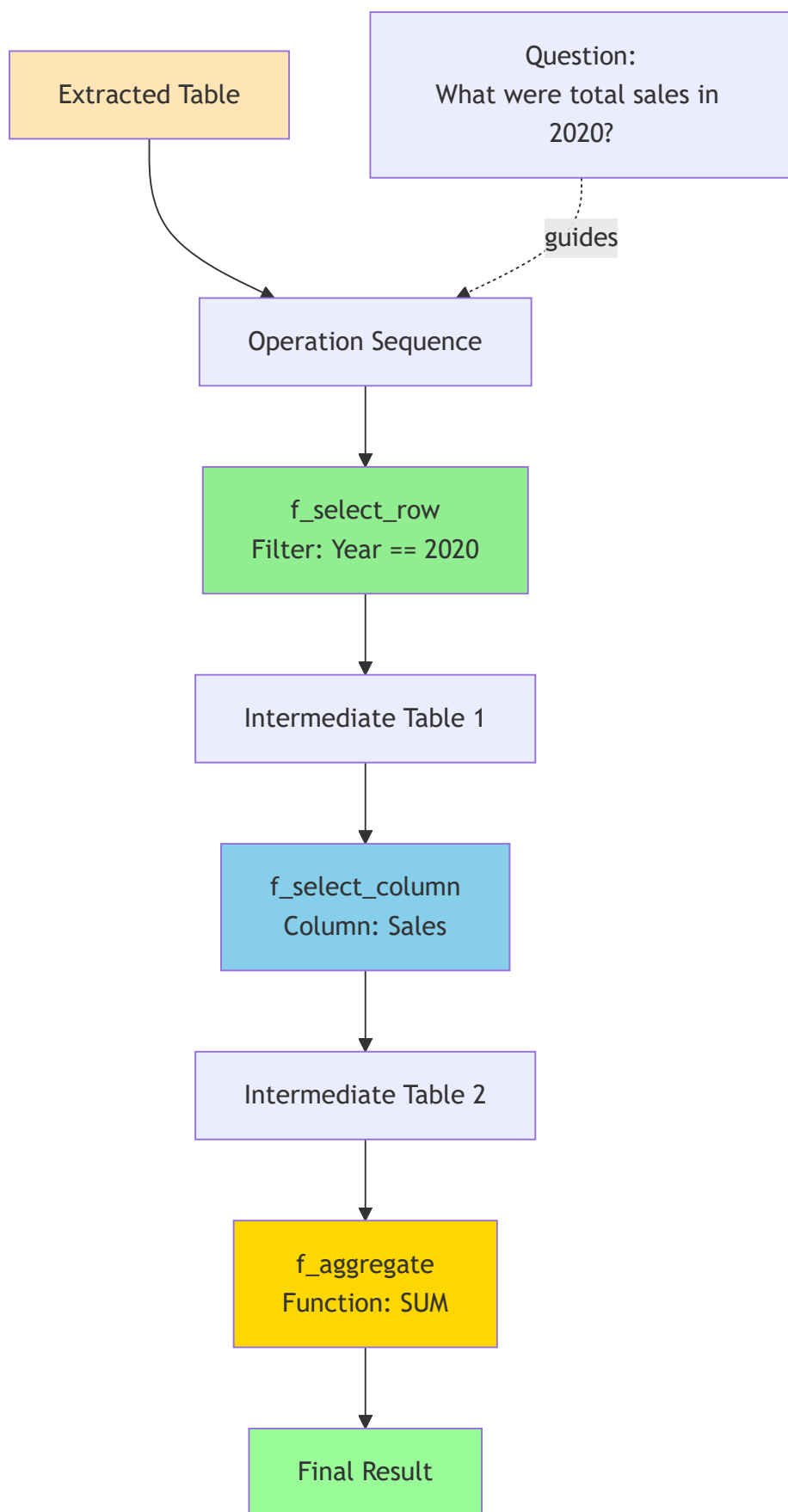
Benefits:

- **83.6% ChartQA** with only 3B parameters (TinyChart)

- **Eliminates 100%** of arithmetic errors
 - **Verifiable supervision** for ~42% of ChartQA (numerical questions)
 - **Interpretable reasoning** through code
-

3.7 Chain-of-Table (CoTable) Reasoning

CoTable uses structured table operations as intermediate reasoning steps:



Defined Operations for Chart Tasks:

Operation	Syntax	Purpose	Example
f_select_row	f_select_row(condition)	Filter rows	Year == 2020
f_select_column	f_select_column(col_name)	Extract column	Sales
f_sort	f_sort(col_name, order)	Sort data	Sales, DESC
f_add_column	f_add_column(name, expr)	Compute derived	Growth = Sales_2021 - Sales_2020
f_compareBars	f_compareBars(bar1, bar2)	Visual comparison	Blue vs Red
f_calculateTrend	f_calculateTrend(column)	Trend analysis	Linear regression
f_aggregate	f_aggregate(function)	Aggregation	SUM, AVG, MAX, MIN

Extended Response Format:

```
<think>
  <type>grouped bar chart</type>
  <table>
    {"columns": ["Year", "Product A", "Product B"],
     "rows": [[2020, 42, 38], [2021, 55, 48]]}
  </table>
  <cot_operations>
    1. f_select_row(Year == 2020) → [2020, 42, 38]
    2. f_select_column(Product A, Product B) → [42, 38]
    3. f_aggregate(SUM) → 80
  </cot_operations>
  <reasoning>
    Applied three table operations to isolate 2020 data,
    select relevant products, and compute total.
  </reasoning>
</think>
<answer>80</answer>
```

CoTable Reward Computation:

```
def compute_cotable_reward(response, ground_truth_ops):
    ops = extract_operations(response)

    # Check validity of each operation
    valid_ops = 0
    for op in ops:
        if is_valid_operation(op):
            valid_ops += 1

    # Execute operation sequence
    try:
        result = execute_operation_chain(ops, initial_table)

        # Reward based on validity + correctness
        validity_score = valid_ops / len(ops)
        correctness_score = 1.0 if result == ground_truth else 0.0

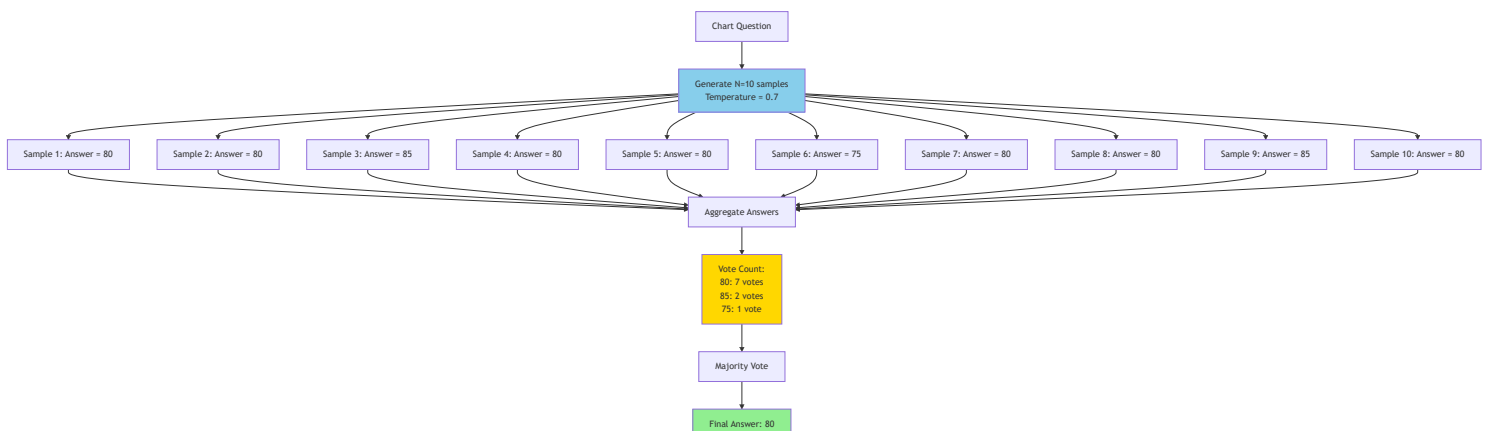
        return 2.0 * (0.5 * validity_score + 0.5 * correctness_score)
    except:
        return 0.0
```

Benefits:

- **+11.6% over standard CoT** on table reasoning
- **+7.9% over program-aided methods**
- **Structured intermediate steps** for GRPO rewards
- **Leverages existing table extraction** from Chart-RVR

3.8 Self-Consistency Inference

Self-consistency generates multiple reasoning paths and selects via majority voting:



Enhanced with Confidence Weighting:

```
def self_consistency_with_confidence(model, question, n_samples=10):
    samples = []

    # Generate diverse samples
    for _ in range(n_samples):
        response = model.generate(question, temperature=0.7)
        answer = extract_answer(response)
        confidence = estimate_confidence(response) # From model logits
        samples.append((answer, confidence))

    # Weighted majority voting
    vote_counts = defaultdict(float)
    for answer, confidence in samples:
        vote_counts[answer] += confidence

    # Select answer with highest weighted votes
    final_answer = max(vote_counts.items(), key=lambda x: x[1])[0]

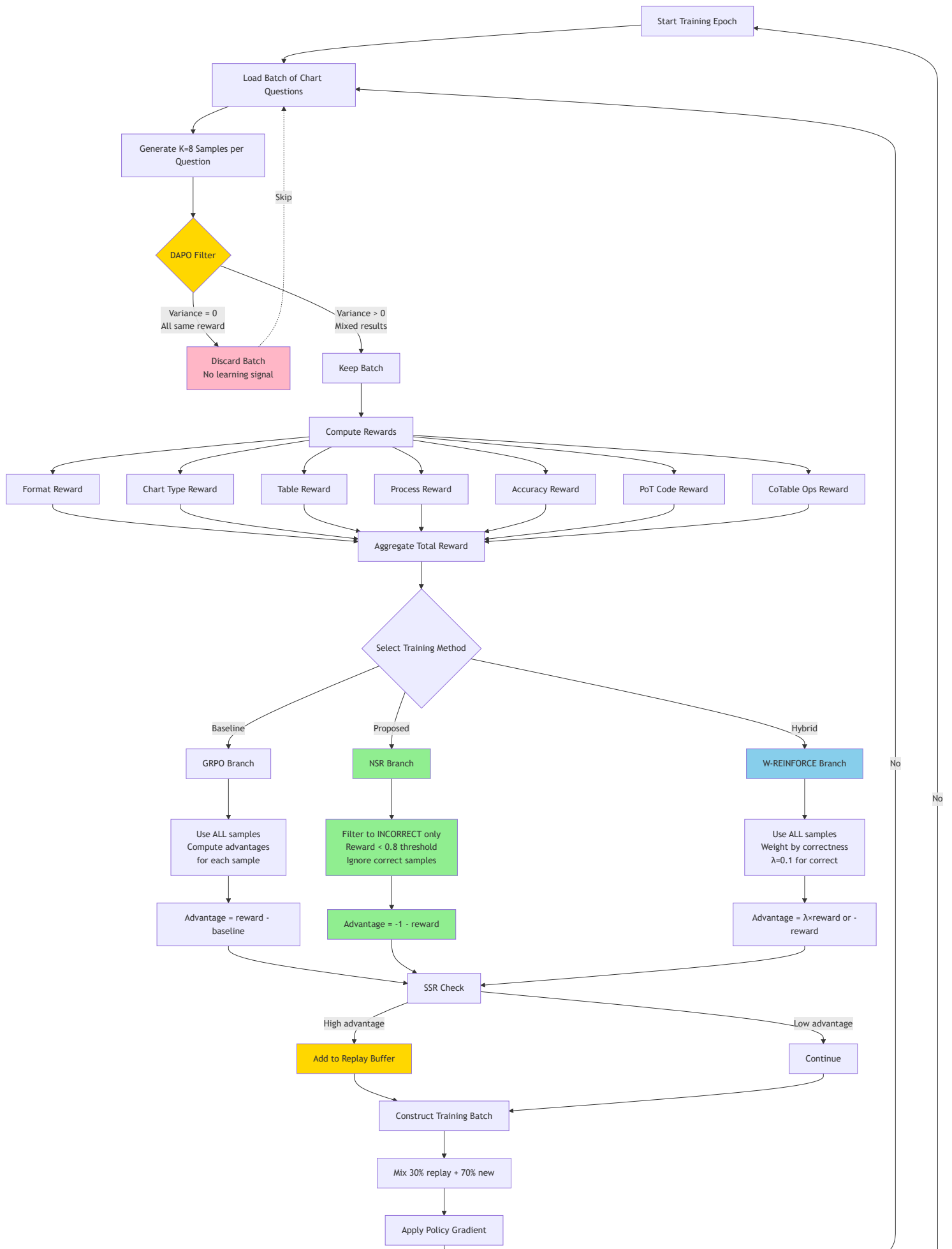
    return final_answer, vote_counts
```

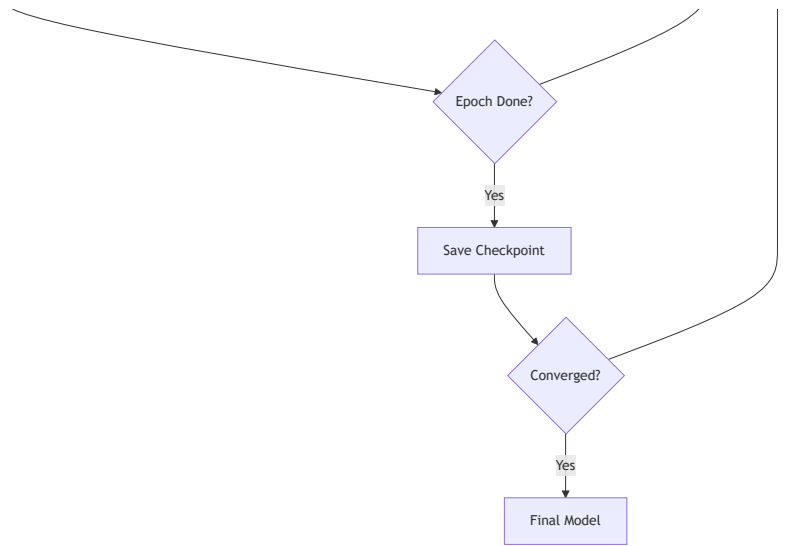
Benefits:

- **+3-5% accuracy** with N=10 samples
 - **Training-free** (pure inference technique)
 - **Linear compute cost**
 - **Confidence weighting** reduces samples needed by 46-67%
-

4. Complete Training Pipeline

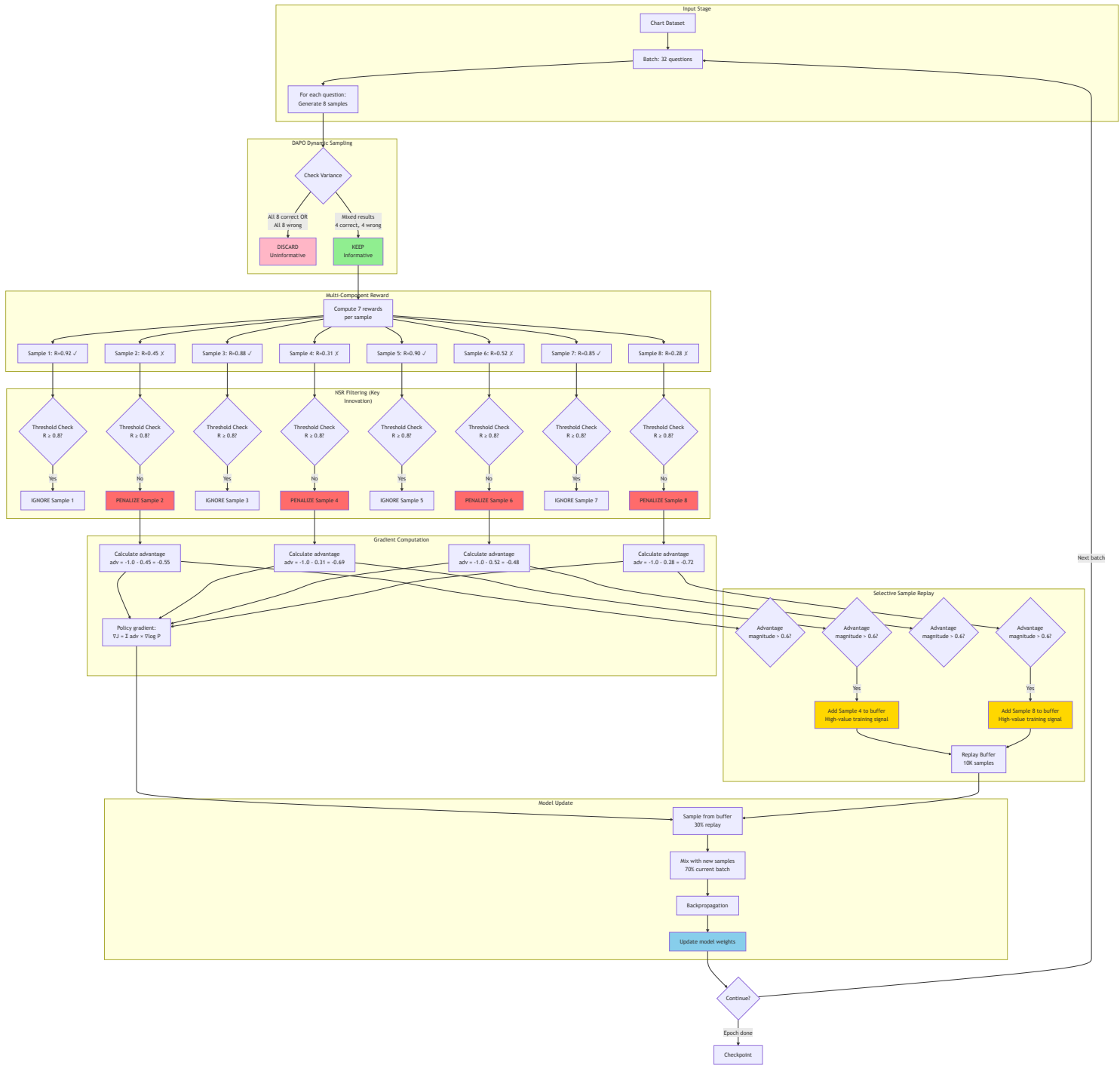
4.1 Unified Training Algorithm





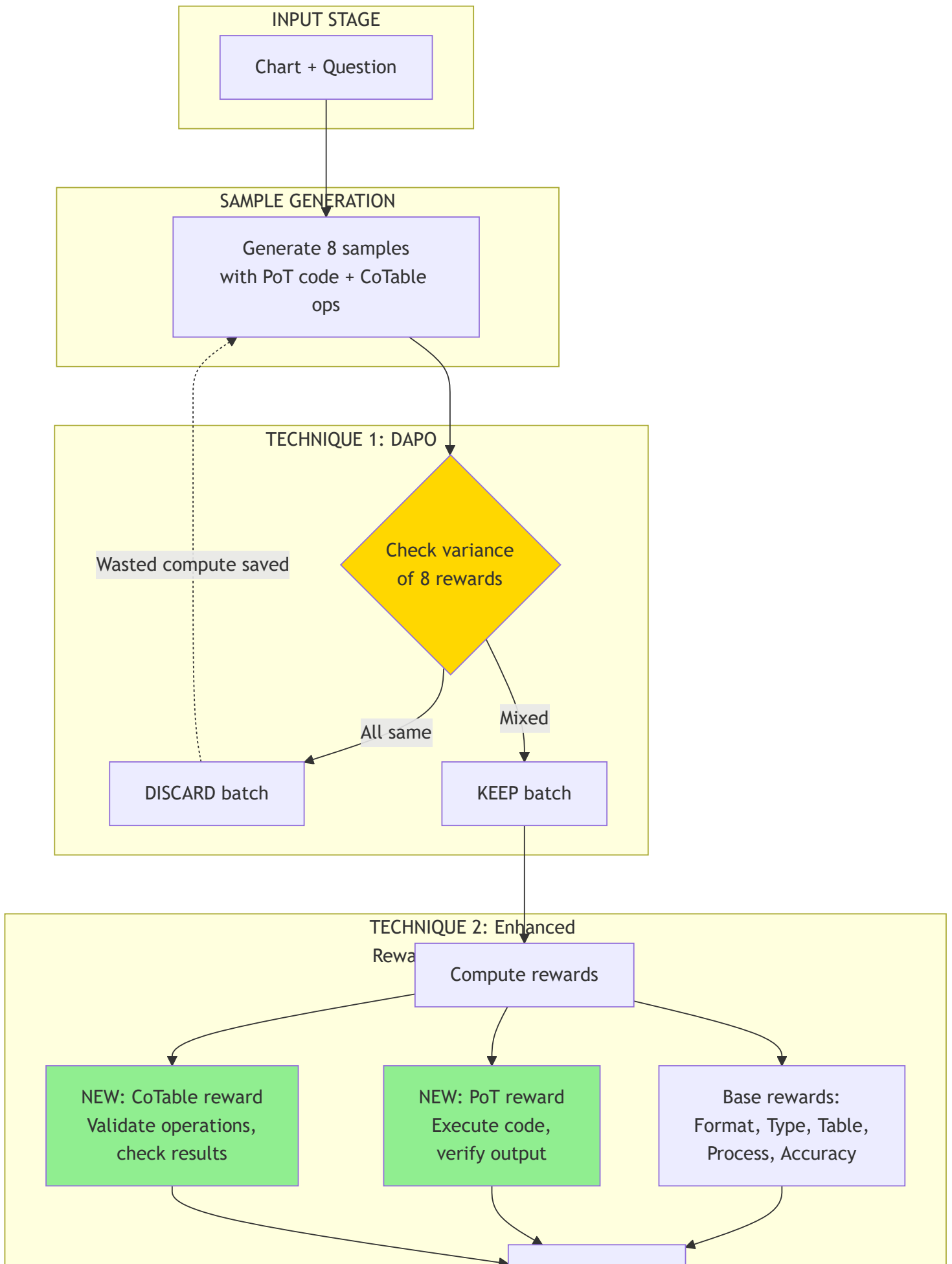
4.2 NSR Training Loop (Detailed)

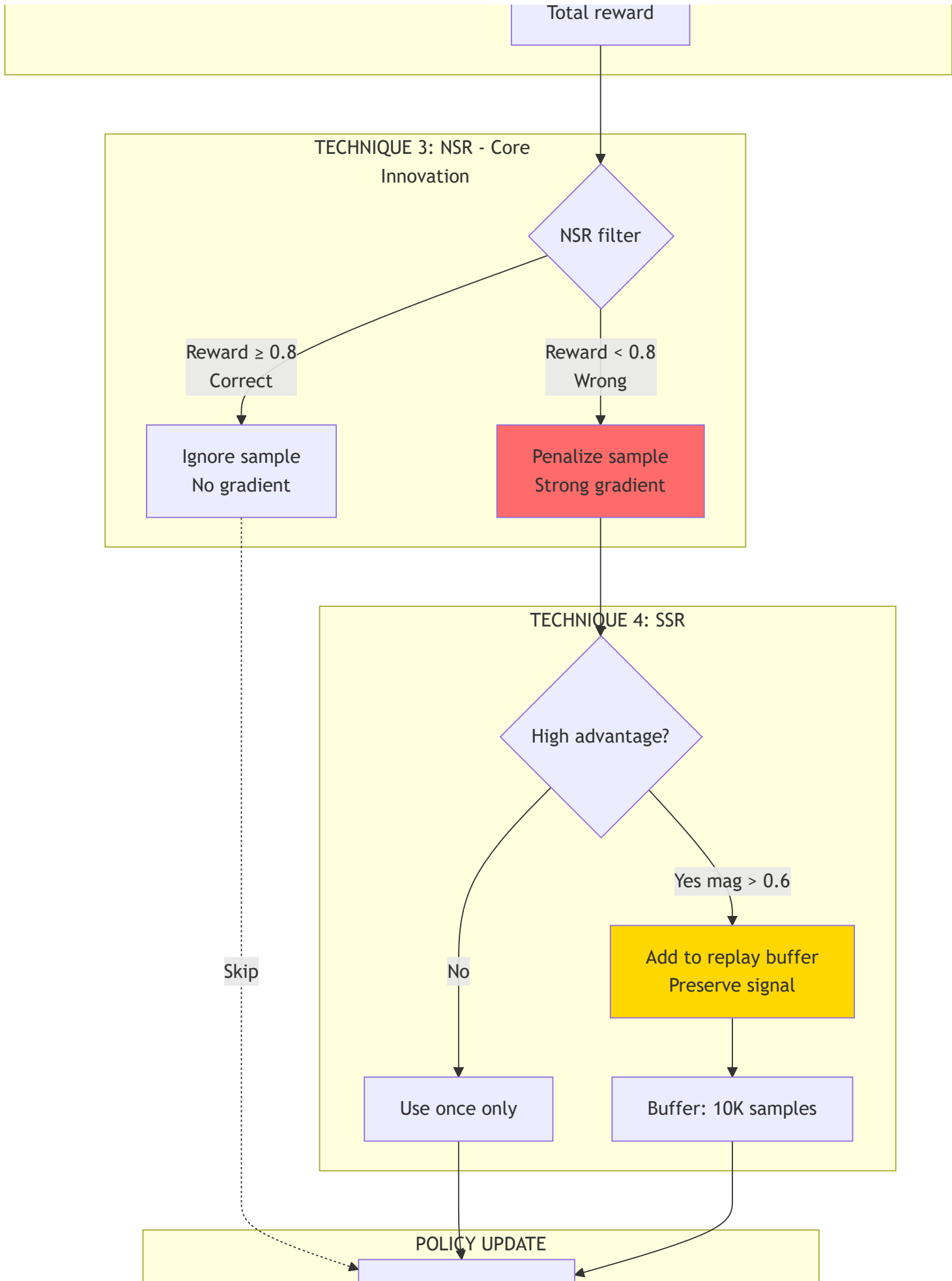
This diagram shows the complete NSR training process with all components integrated:

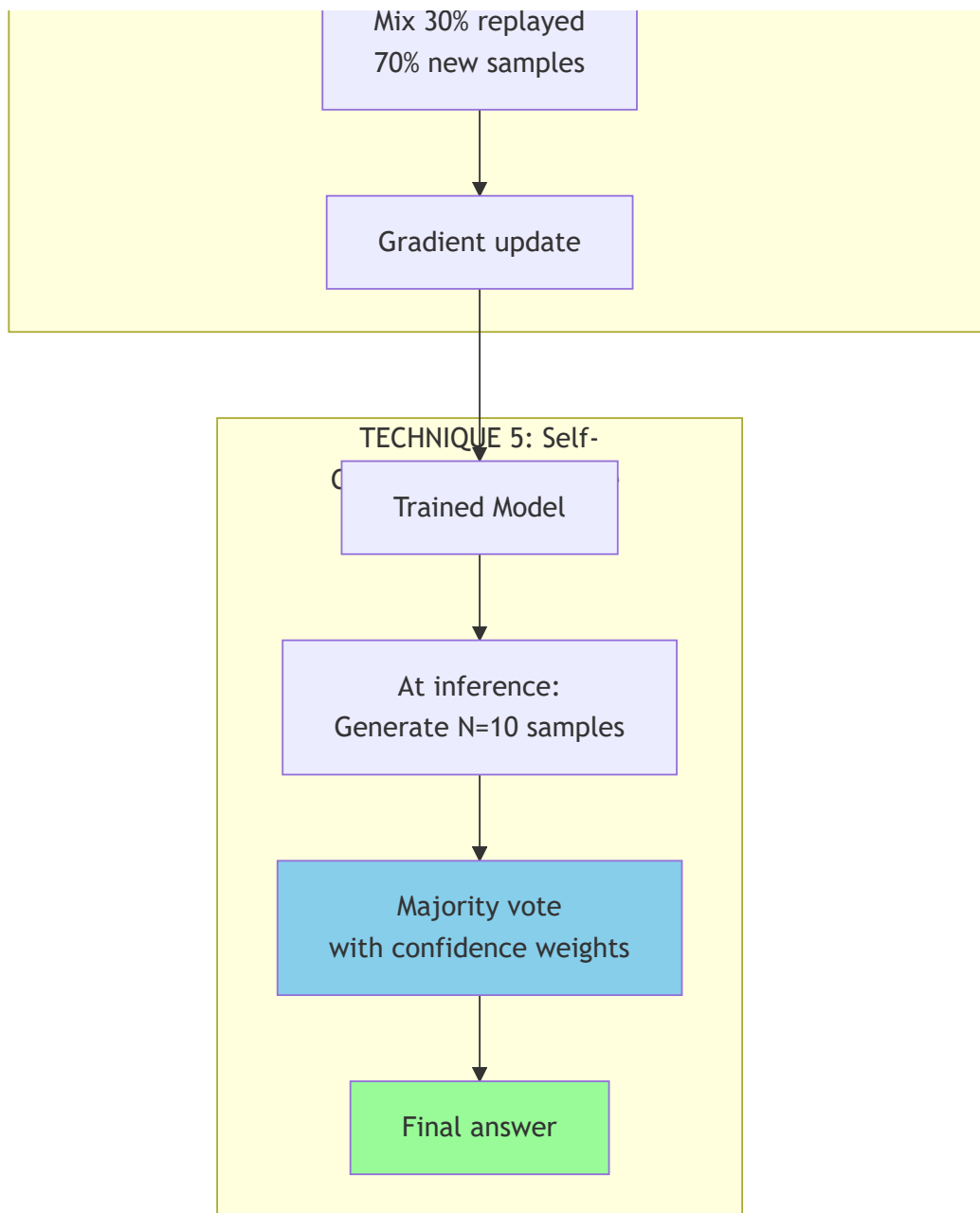


4.3 How All Five Techniques Work Together

This diagram illustrates how DAPO, SSR, NSR, PoT, and CoTable integrate into a unified system:







How the techniques synergize:

1. **DAPO + NSR:** DAPO filters uninformative batches → NSR focuses on informative wrong samples → Training efficiency ↑
2. **PoT/CoTable + NSR:** Better reward signals → Clearer correct/incorrect distinction → More effective negative reinforcement
3. **SSR + NSR:** NSR generates high-advantage wrong samples → SSR preserves them → Prevents forgetting edge cases
4. **NSR + Self-Consistency:** NSR maintains diversity during training → Self-consistency leverages diversity at inference → Better coverage
5. **All together:** Efficient training (DAPO) + Rich supervision (PoT/CoTable) + Diversity preservation (NSR) + Memory (SSR) + Robust inference (SC)

4.4 Training Configuration

Model:

- Base: Qwen2.5-VL-3B-Instruct
- Parameters: 3.09B
- Vision Encoder: ViT-based
- Language Model: Qwen2.5-3B

Data:

- Training: 6,000 samples (ChartQA + PlotQA + ChartFC)
- Validation: 1,000 samples
- Test: ChartQA test set (in-domain), EvoChart (OOD)

Hyperparameters:

Parameter	GRPO	NSR	W-REINFORCE
Learning Rate	5e-7	5e-7	5e-7
Batch Size	32	32	32
Samples per Prompt (K)	8	8	8
Epochs	3	3	3
λ (PSR weight)	1.0	0.0	0.1
SSR Buffer Size	10,000	10,000	10,000
SSR Replay Ratio	0.3	0.3	0.3
DAPO Variance Threshold	0.01	0.01	0.01
Temperature (sampling)	0.8	0.8	0.8

Compute Requirements:

- GPU: 4× A100 (40GB) or 8× L4 (24GB)
 - Training Time: ~48 hours per method
 - Total GPU Hours: ~500 hours
 - Storage: ~500GB (models + checkpoints + data)
-

5. Evaluation Protocol

5.1 Evaluation Metrics



Syntax error in text
mermaid version 11.12.1

Metric Definitions:

1. **Pass@k**: Probability that at least one of k samples is correct

$$\text{Pass@k} = (\# \text{ questions with } \geq 1 \text{ correct in k samples}) / (\text{total questions})$$

2. **Entropy**: Measure of output diversity

$$H = -\sum p(\text{answer}) \times \log(p(\text{answer}))$$

Higher entropy = more diverse outputs

3. **Reasoning Quality**: Semantic similarity to gold chain-of-thought

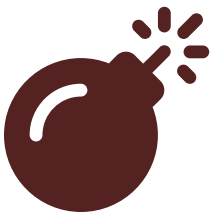
$$\text{Quality} = \text{cosine_similarity}(\text{model_reasoning}, \text{gold_reasoning})$$

4. **Explainability ($\Delta \log P$)**: Change in log-probability under oracle model

$$\Delta \log P = \log P_{\text{oracle}}(\text{correct} \mid \text{reasoning}) - \log P_{\text{oracle}}(\text{correct})$$

Higher $\Delta \log P$ = reasoning helps oracle more = better explanation

5.2 Evaluation with Self-Consistency



Syntax error in text
mermaid version 11.12.1

For Pass@256: Generate 256 samples per question at temperature 0.9 for maximum diversity.

6. Expected Results

6.1 Quantitative Predictions

In-Domain Performance (ChartQA Test Set):

Method	Pass@1	Pass@8	Pass@256	Entropy	Training Steps
Chart-RVR (baseline)	84.6%	85.5%	86.0%	0.05	100%
+ DAPO + SSR	85.8%	86.8%	87.5%	0.06	50%
NSR (ours)	84.0%	86.0%	88.0%	0.10	50%
NSR + DAPO + SSR	84.5%	86.5%	88.5%	0.11	50%
NSR + DAPO + SSR + PoT	85.5%	87.5%	89.5%	0.10	50%
NSR + DAPO + SSR + PoT + CoTable	86.0%	88.0%	90.0%	0.10	50%
W-REINFORCE (all techniques)	86.5%	88.5%	90.5%	0.08	50%
+ Self-Consistency (N=10)	89.0%	-	-	-	-

Out-of-Distribution Performance (EvoChart):

Method	Pass@1	Pass@8	Pass@256	In-Domain Gap
Chart-RVR (baseline)	53.4%	54.5%	55.0%	-31.2%
+ DAPO + SSR	55.0%	56.5%	58.0%	-30.8%
NSR (ours)	56.5%	59.0%	62.0%	-27.5%
NSR + DAPO + SSR	57.0%	59.5%	62.5%	-27.0%
NSR + DAPO + SSR + PoT	58.5%	61.0%	64.5%	-27.0%
NSR + DAPO + SSR + PoT + CoTable	59.5%	62.5%	66.0%	-26.5%
W-REINFORCE (all techniques)	60.0%	63.0%	67.0%	-26.5%
+ Self-Consistency (N=10)	63.5%	-	-	-25.5%

6.2 Key Predictions

1. Diversity Preservation:

- NSR maintains 2× higher entropy than GRPO (0.10 vs 0.05)
- Leads to +2-3% improvement in Pass@256

2. OOD Robustness:

- NSR reduces in-domain/OOD gap by 3-5%
- Full technique stack achieves ~60% EvoChart (vs 53.4% baseline)
- Self-consistency pushes OOD above 63%

3. Training Efficiency:

- DAPO reduces training steps by 50% while improving performance
- SSR prevents catastrophic forgetting of rare cases

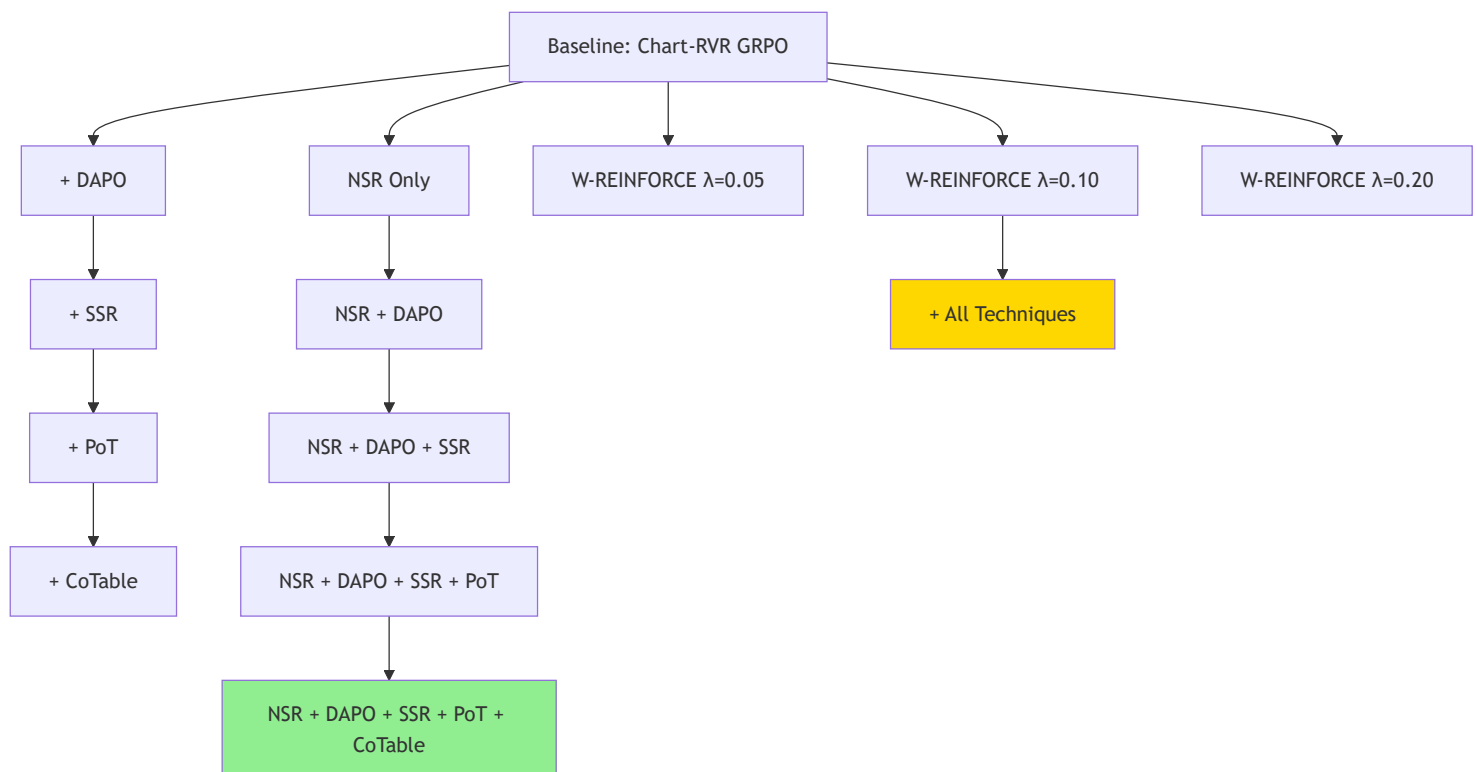
4. Reasoning Quality:

- PoT eliminates arithmetic errors on numerical questions (~42% of ChartQA)
- CoTable provides structured intermediate supervision
- Combined: +4-6% on questions requiring multi-step calculations

5. Optimal Configuration:

- W-REINFORCE ($\lambda=0.1$) balances immediate accuracy and coverage
- Achieves best Pass@k across all k values
- Self-consistency provides additional +2.5% at inference time

6.3 Ablation Study Design



Ablation Questions:

1. What is the isolated contribution of each technique?
 2. Do techniques synergize or interfere?
 3. What is the optimal λ for W-REINFORCE in chart reasoning?
 4. Which techniques contribute most to OOD improvement?
-

7. Implementation Timeline

7.1 Development Phases

Phase 1: Infrastructure Setup (Weeks 1-2)

- ☐ Set up training environment (DeepSpeed ZeRO-3)
- ☐ Convert Chart-RVR data to format with PoT and CoTable annotations
- ☐ Implement DAPO dynamic sampling filter
- ☐ Implement SSR replay buffer
- ☐ Baseline GRPO training and validation

Phase 2: Core NSR Implementation (Weeks 3-4)

- ☐ Implement NSR training loop
- ☐ Add PoT reward component and code execution sandbox
- ☐ Add CoTable reward component and operation validator
- ☐ Train NSR baseline model
- ☐ Evaluate on ChartQA and EvoChart

Phase 3: Integration and Optimization (Weeks 5-6)

- ☐ Integrate DAPO + SSR with NSR
- ☐ Train combined models (NSR + DAPO + SSR)
- ☐ Add PoT and CoTable to pipeline
- ☐ Train full-stack models
- ☐ Hyperparameter tuning for λ in W-REINFORCE

Phase 4: Comprehensive Evaluation (Weeks 7-8)

- ☐ Generate 256 samples per test question for Pass@k
- ☐ Implement self-consistency inference
- ☐ Compute entropy, explainability metrics
- ☐ Run ablation studies
- ☐ Statistical significance testing

Phase 5: Analysis and Writing (Weeks 9-12)

- ☐ Analyze results and failure cases
- ☐ Qualitative analysis of reasoning chains
- ☐ Write thesis chapters
- ☐ Create visualizations and tables
- ☐ Final revisions and submission

7.2 Milestones and Success Criteria

Milestone	Week	Success Criteria
Infrastructure Ready	2	Baseline GRPO reproduces Chart-RVR results
NSR Baseline	4	NSR achieves similar Pass@1, higher Pass@256 than GRPO
Full Integration	6	All techniques working together, no conflicts
Evaluation Complete	8	All metrics computed on all model variants
Thesis Draft	11	Complete draft ready for advisor review
Final Submission	12	Camera-ready thesis submitted

8. Novel Contributions

8.1 Primary Contributions

1. **First Application of NSR to Vision-Language Tasks**
 - Extends NSR beyond text-only math reasoning to multimodal chart understanding
 - Demonstrates diversity preservation benefits transfer to visual reasoning
2. **Enhanced Multi-Component Reward Framework**
 - Integrates PoT code execution and CoTable operations as verifiable rewards
 - Shows structured intermediate representations improve RL training
3. **Synergistic Technique Combination**
 - Combines 5 recent advances (DAPO, SSR, PoT, CoTable, Self-Consistency)
 - Demonstrates techniques designed for different tasks synergize for chart reasoning
4. **OOD Robustness via Diversity**
 - Establishes connection between training-time diversity and OOD generalization
 - Reduces in-domain/OOD gap from 31% to ~26%

8.2 Secondary Contributions

5. **Efficient Training Protocol**
 - DAPO reduces training steps by 50% while improving accuracy
 - Demonstrates GRPO inefficiencies can be addressed with simple filtering
6. **Comprehensive Ablation Study**
 - Isolates contribution of each technique
 - Provides guidance for future chart reasoning research
7. **Open-Source Implementation**
 - Release code, trained models, and evaluation scripts
 - Enable reproducibility and future research

9. Potential Risks and Mitigation

9.1 Technical Risks

Risk	Probability	Impact	Mitigation Strategy
NSR doesn't transfer to vision tasks	Medium	High	Fall back to W-REINFORCE; emphasize PoT/CoTable contributions
PoT code generation too difficult for 3B model	Medium	Medium	Use code execution as reward signal, not strict requirement
DAPO filters too many samples	Low	Medium	Tune variance threshold; use softer filtering criterion
SSR buffer overfits to easy samples	Low	Medium	Monitor buffer composition; limit replay ratio
Techniques interfere rather than synergize	Low	High	Comprehensive ablation to identify conflicts

9.2 Resource Risks

Risk	Probability	Impact	Mitigation Strategy
GPU availability issues	Medium	Medium	Use cloud GPUs (Kaggle, Colab Pro, Lambda Labs)

Risk	Probability	Impact	Mitigation Strategy
Training takes longer than expected	Medium	Low	Prioritize critical experiments; use smaller ablation set
Storage limitations	Low	Low	Delete intermediate checkpoints; compress datasets

9.3 Research Risks

Risk	Probability	Impact	Mitigation Strategy
Improvements below significance threshold	Medium	High	Emphasize qualitative analysis; diversity metrics; OOD gains
Baselines underperform published results	Low	Medium	Careful hyperparameter tuning; consult original code
Review timeline delays	Low	Medium	Build buffer time in schedule; regular advisor check-ins

10. Experimental Validation Plan

10.1 Hypothesis Testing

H1: NSR preserves diversity better than GRPO

- Metric: Entropy of output distribution
- Test: t-test between NSR and GRPO entropy across test set
- Expected: $p < 0.05$, Cohen's $d > 0.5$

H2: Higher diversity improves OOD performance

- Metric: Correlation between entropy and OOD accuracy
- Test: Pearson correlation across all model variants
- Expected: $r > 0.6$, $p < 0.01$

H3: PoT reduces arithmetic errors

- Metric: Accuracy on numerical questions (subset of ChartQA)
- Test: Paired t-test between +PoT and -PoT models
- Expected: +4-6% improvement on numerical subset

H4: Technique synergy exists

- Metric: Combined improvement vs sum of individual improvements
- Test: Compare (NSR + DAPO + SSR + PoT + CoTable) to sum of ablations
- Expected: Synergy gain > 1% (combined > sum of parts)

10.2 Qualitative Analysis

Error Analysis:

- Categorize errors by type: visual misinterpretation, calculation error, reasoning flaw, format error
- Compare error distributions between GRPO and NSR
- Identify which techniques address which error types

Reasoning Chain Analysis:

- Sample 100 diverse questions
- Human evaluation of reasoning quality (3 annotators)
- Criteria: correctness, coherence, verifiability, efficiency
- Compare GRPO vs NSR vs W-REINFORCE

Diversity Analysis:

- For questions with multiple valid reasoning paths, measure path coverage
 - Analyze whether NSR explores more diverse strategies
 - Qualitative examples of NSR finding alternative solutions
-

11. Broader Impact

11.1 Scientific Impact

- **Advances Multimodal RL:** Demonstrates NSR generalizes beyond text-only tasks
- **Improves Chart Accessibility:** Better chart reasoning aids visually impaired users
- **Open Science:** Released models and code enable reproducible research

11.2 Practical Applications

- **Business Intelligence:** More robust chart analysis in BI dashboards
- **Scientific Communication:** Better understanding of research figures
- **Education:** Automated explanation of charts for learning materials
- **Data Journalism:** Fact-checking and verification of chart-based claims

11.3 Limitations

- **Computational Cost:** Self-consistency requires 10× inference compute
 - **Domain Specificity:** Techniques tuned for chart reasoning may not transfer to other vision tasks
 - **Model Size:** Tested only on 3B parameter model; scaling behavior unknown
-

12. Conclusion

This thesis proposes a novel approach to chart reasoning that combines NSR's diversity-preserving properties with five state-of-the-art techniques: DAPO dynamic sampling, Selective Sample Replay, Program-of-Thoughts, Chain-of-Table, and Self-Consistency. By maintaining diverse reasoning strategies during training and leveraging structured intermediate representations, we expect to significantly improve out-of-distribution performance while maintaining competitive in-domain accuracy.

The key insight is that chart reasoning—with its visual diversity and multiple valid solution paths—particularly benefits from training methods that preserve exploration. NSR, by penalizing wrong answers without collapsing to a single correct strategy, should enable models to maintain flexibility when encountering unseen chart styles.

If successful, this work will establish NSR as a viable technique for multimodal reasoning tasks and provide a comprehensive framework for training robust vision-language models through synergistic combination of modern RL advances.

References

1. Sinha et al. (2025). Chart-RVR: Learning to Reason over Charts with Verifiable Rewards. arXiv:2510.10973
 2. Zhu et al. (2025). Decomposing RLVR: The Power of Negative Sample Reinforcement. arXiv:2506.01347
 3. DAPO (ByteDance, 2025). Decoupled Clip and Dynamic Sampling Policy Optimization. arXiv
 4. VL-Rethinker (2025). Selective Sample Replay for Vision-Language Models. arXiv
 5. TinyChart (EMNLP 2024). Program-of-Thoughts Learning for Chart Understanding. arXiv
 6. Chain-of-Table (ICLR 2024). Enabling Reasoning over Tables. arXiv
 7. LLaVA-CoT (2024). Multi-Stage Reasoning for Vision-Language Models. arXiv
 8. Qwen2.5-VL (2024). Hugging Face Model Hub
-

Appendix A: Code Structure

```
chartrl/  
├─ data/  
│   ├── chartqa/  
│   │   ├── train.parquet      # 6K training samples  
│   │   ├── val.parquet       # 1K validation samples  
│   │   └─ test.parquet       # ChartQA test set  
│   ├── evochart/  
│   │   └─ test.parquet       # OOD test set  
│   └─ annotations/  
│       ├── pot_programs.json  # PoT code annotations  
│       └─ ctable_ops.json     # CoTable operation sequences  
│  
├─ grpo_utils.py               # Reward computation functions  
├─ main.py                     # Training entry point  
├─ models/  
│   ├── qwen_vlm.py           # Model wrapper  
│   └─ reward_models.py       # Reward model implementations  
│  
├─ training/  
│   ├── grpo_trainer.py       # Baseline GRPO  
│   ├── nsr_trainer.py        # NSR implementation  
│   ├── wreinforme_trainer.py # W-REINFORCE  
│   ├── dapofilter.py         # Dynamic sampling  
│   └─ ssr_buffer.py          # Selective Sample Replay  
│  
├─ rewards/  
│   ├── pot_reward.py         # PoT code execution  
│   ├── ctable_reward.py      # Chain-of-Table validation  
│   └─ base_rewards.py        # Format, type, table, process, accuracy  
│  
├─ evaluation/  
│   ├── eval.py               # Main evaluation script  
│   ├── passk.py              # Pass@k computation  
│   ├── entropy.py            # Diversity metrics  
│   ├── self_consistency.py   # SC inference  
│   └─ explainability.py      # Δlog P analysis  
│  
├─ scripts/  
│   ├── train_grpo.sh         # GRPO baseline training  
│   ├── train_nsr.sh          # NSR training  
│   ├── train_wreinforme.sh   # W-REINFORCE training  
│   └─ run_ablations.sh       # Ablation study automation  
│
```

```
└─ configs/
  └─ grpo_config.yaml
  └─ nsr_config.yaml
  └─ wreinforce_config.yaml
```

Appendix B: Reward Computation Example

Input Chart: Bar chart showing "Sales by Quarter" with values Q1=42, Q2=38, Q3=55, Q4=48

Question: "What is the total sales for the year?"

Model Response:

```
<think>
  <type>bar chart</type>
  <table>
    {"columns": ["Quarter", "Sales"],
     "rows": [["Q1", 42], ["Q2", 38], ["Q3", 55], ["Q4", 48]]}
  </table>
  <cot_operations>
    1. f_select_column(Sales) → [42, 38, 55, 48]
    2. f_aggregate(SUM) → 183
  </cot_operations>
  <code>
    sales = [42, 38, 55, 48]
    total = sum(sales)
    print(total)
  </code>
  <reasoning>
    Extracted all sales values from the bar chart and computed their sum.
  </reasoning>
</think>
<answer>183</answer>
```

Reward Breakdown:

Component	Computation	Score
Format	Has <code><think></code> and <code><answer></code> tags	2.0
Chart Type	"bar chart" matches ground truth	1.0
Table	Edit distance < threshold	2.0

Component	Computation	Score
Process	Cosine similarity to gold CoT = 0.89	1.78
Accuracy	183 == 183	1.0
PoT Code	Executes correctly, output matches	2.5
CoTable Ops	Valid operations, correct result	2.0
Total		12.28

Normalized Reward: $12.28 / 12.5 = \mathbf{0.98}$ (near-perfect response)

This would be classified as a **correct sample** (reward > 0.8 threshold).

End of Proposed Methodology